

Trabalho Prático de Sistemas Multimédia

JOÃO PAULO MARTINS NOVO

3ºAno - ECGM

Índice

Objetivo e descrição do trabalho	3
Recolha de dados e ideia	3
Escolas do IPVC	3
Propostas de jogos	4
1. Jogo de Tapetes Rolantes (Possível para ESE; ESA; ESS)	4
2. Jogo de Avião (Possível para ESDL)	4
3. Chuva de Objetos Educativos (ESE – Licenciatura em Educação)	5
4. Simulação de Enfermagem (ESS – Licenciatura em Enfermagem)	5
Funcionalidades Desenvolvidas	5
Cumprimento dos requisitos	6
Classe Jogo	8
Classe Inimigo	10
Classe Player	11
Classe Seringa	12
Referências	13
Conclusão	14

Objetivo e descrição do trabalho

O objetivo deste trabalho prático foi criar um jogo educacional, utilizando recursos de áudio e elementos interativos via audio + rato/teclado, relacionado a uma das escolas do Instituto Politécnico de Viana do Castelo (IPVC).

Link Github: <https://github.com/shoteose/NurseSimulatorGame>

Recolha de dados e ideia

Escolas do IPVC

- ESTG – Escola Superior de Tecnologia e Gestão
- ESE – Escola Superior de Educação
- ESA - Escola Superior Agrária
- ESS - Escola Superior de Saúde
- ESCE – Escola Superior de Ciências Empresariais
- ESDL - Escola Superior de Desporto e Lazer

Propostas de jogos

As ideias para o desenvolvimento dos jogos foram concebidas considerando as diversas escolas do IPVC. Embaixo estão as propostas de jogos desenvolvidas para cada escola:

1. Jogo de Tapetes Rolantes (Possível para ESE; ESA; ESS)

- **Descrição Geral:** Este jogo consiste em três tapetes rolantes pelos quais passam diversos tipos de objetos, alguns bons outros maus. O jogador deve apanhar os objetos corretos, o que aumenta a pontuação, enquanto evitar os objetos prejudiciais preserva a vida do personagem.
- **Mecânica de Jogo:**
 - **Controlo do Jogador:** O áudio controla qual tapete o player está. O jogador usa o rato ou teclado para apanhar ou evitar os objetos que passam nos tapetes.
 - **Sistema de Bónus:** Ao apanhar corretamente uma sequência de objetos bons, aumenta o multiplicador.

2. Jogo de Avião (Possível para ESDL)

- **Descrição Geral:** Um jogo onde o jogador controla um avião em altura, enfrentando inimigos enquanto sobe e desce.
- **Mecânica de Jogo:**
 - **Controlo do Avião:** O áudio controla a subida e descida do avião, enquanto o jogador dispara contra os inimigos utilizando o rato ou o teclado.
 - **Sistema de Bónus:** Ao derrubar corretamente uma sequência de aviões, aumenta o multiplicador.

3. Chuva de Objetos Educativos (ESE – Licenciatura em Educação)

- **Descrição Geral:** Um jogo de recolha de objetos em queda, onde o jogador deve apanhar os itens corretos com base em uma instrução apresentada no ecrã.
- **Mecânica de Jogo:**
 - **Controlo do Jogador:** O jogador move-se com o áudio para apanhar os objetos corretos, enquanto evita os errados. Ativa o poder com o rato.
 - **Poder:** Existe um escudo, que protege o jogador de objetos.

4. Simulação de Enfermagem (ESS – Licenciatura em Enfermagem)

- **Descrição Geral:** Um jogo onde o jogador assume o papel de um enfermeiro/a onde tem que “vacinar” pessoas que vão ficando doentes ao longo do tempo.
- **Mecânica de Jogo:**
 - **Controlo do Jogador:** O jogador deve deslocar-se pelo mapa para vacinar as pessoas antes que a sua condição se agrave. O áudio é utilizado para realizar a ação de vacinar.

[Vários tipos de vacinas/ doenças] -> Ideia fornecida pelo professor na
Apresentação Individual da Ideia dia 15 de Outubro

Funcionalidades Desenvolvidas

O jogo selecionado para desenvolvimento foi a Simulação de Enfermagem, em que chamei de “Nurse Simulator Game”. As principais funcionalidades foram:

- **Movimentação e Animação:** O personagem se movimenta nas 8 direções (usando as teclas WASD) com animações correspondentes às 4 direções.
- **Doenças e Vacinas:** Foram criados três tipos de doenças, cada uma com uma seringa correspondente. O player precisa selecionar a seringa conforme a doença.
- **Background Personalizado:** O fundo do jogo foi desenhado para atender ao requisito da nº2.
- **Som e Interação:** O áudio foi usado para atacar no jogo. Sons de fundo, cura e morte dos inimigos também foram implementados.
- **Aleatoriedade Controlada:** O aparecimento de inimigos e o desenvolvimento de doenças são aleatórios, com intervalos de tempo definidos programaticamente (entre 5 a 10 segundos).

Cumprimento dos requisitos

1. O jogo é original e tem como objetivo estabelecido que é ganhar pontos.
2. O jogo faz referência a Escola Superior de Saúde, até mais concretamente á licenciatura de Enfermagem.
3. O meu nome e número de aluno, como a sigla do curso e da cadeira estão dispostos no canto inferior direito no ecrã de entrada do jogo.
4. Quanto a aleatoriedade controlada programaticamente tenho nos inimigos para os iniciar com random do tamanho da tela.

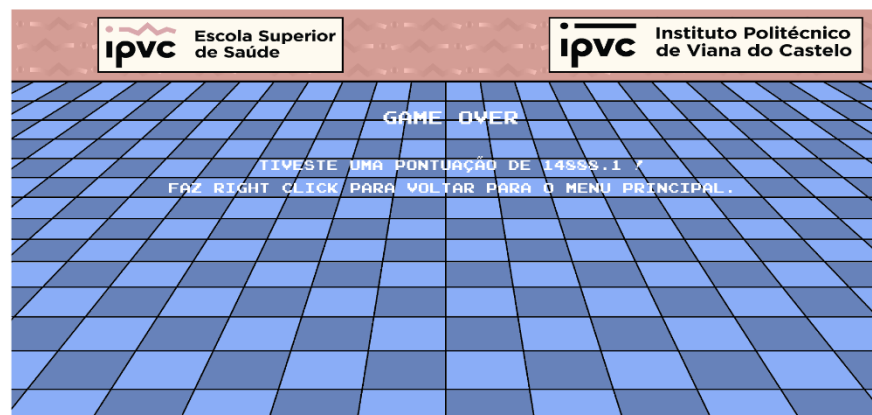
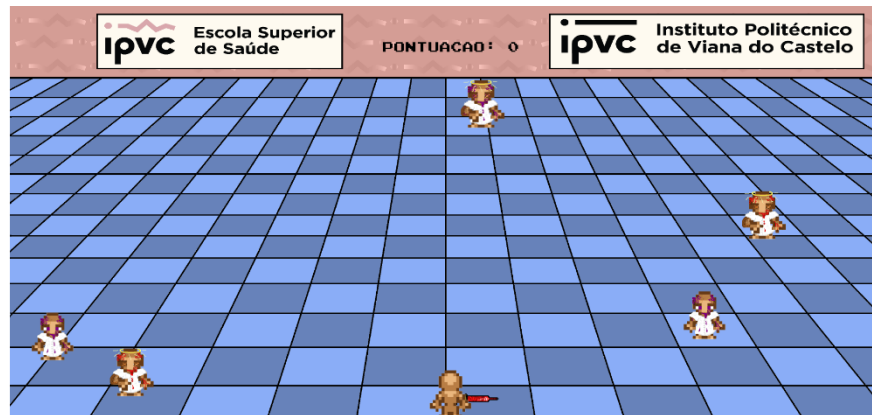
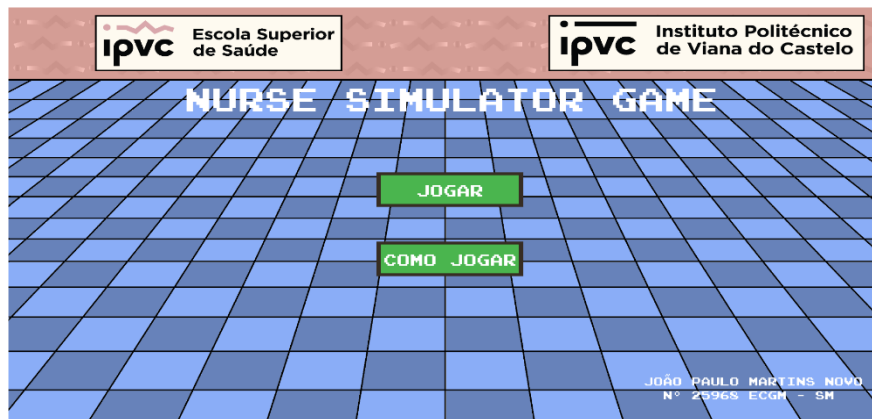
```
function initInimigos() {  
  for (let j = 0; j < nrInimigos; j++) {  
    let pInicialX = random(width);  
    let pInicialY = random(height);  
  
    inim = new Inimigo(pInicialX, pInicialY);  
    inimigos.push(inim);  
  }  
}
```

Fig.1 Método iniciar inimigos

E a própria doença que eles adquiram é randomizada entre as 3 doenças e o seu intervalo de tempo de adoecer também é random entre 5 a 10 segundos.

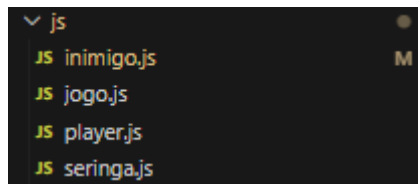
5. O controlo/interação com o jogo é feito com o rato apenas para clicar nos botões, depois na jogabilidade o áudio é medido e passando um certo nível faz com que o player “seringue” os inimigos, o que os faz curar. Depois com o “WASD” e as setinhas faz com que se mova e por fim o “Q,E e R” mudem as seringas para as várias doenças.
6. Com a parede temos o player e os inimigos que refletem, depois tem a seringa com os inimigos.
7. Como som temos a música de fundo, som quando o inimigo é curado, som quando morre e som para o click nos botões.

8. O jogo tem 4 ecrãs:



O ecrã de início, o ecrã que informa como jogar, o ecrã de jogo e ecrã de gameover.

9. O código está estrutura em objetos onde eu tenho a Classe Inimigo, Classe Player e a Classe Seringa onde depois uso na principal.



Classe Jogo

Na classe jogo, a principal temos os seguinte métodos:

- `carregaMedia()`:

Aqui carrego a fonte, alguns sons e imagens necessárias logo de inicio.

- `preload()` [método do p5]:

Evoco o método `carregaMedia()` para carregar já no `preload`.

- `setup()` [método do p5]:

Crio o canvas com o tamanho da janela, defino o frame rate e começo o input para o áudio.

- `initInimigos()`:

Neste método inicio 5 inimigos com posições randomizadas.

- `mouseClicked()` [método do p5]:

Neste método para quando o mouse é clickado, verifica a posição do mouse e se coincidir sobre dois “retângulos”, um faz iniciar o jogo e o outro ver como jogar o jogo.

- `menuInicial()`:

Este método é o que desenha o menu inicial.

- `detetaKeys()`:

Verifica as variáveis estado e conforme evoca o `movePlayer()`.

- `keyPressed()` [método do p5]:

Verifica se clicou nas teclas de andar, mudar de seringa e correspondentemente mudar variáveis, mudar a imagem da seringa, etc.. isto caso não esteja a atacar.

- `keyReleased()` [método do p5]:

Quando as teclas largadas, faz os respetivos estados serem falsos.

- `menuComoJogar()`:

Este método desenha o ecrã que ensina como jogar o jogo.

- `initMusica()`:

Este método inicia a música automaticamente e mete-a em loop.

- `mostraInim()`:

Percorre pelos inimigos e evoca a função `moveInimigo()`.

- `removeInimigo()`:

Este método remove os inimigos do array. Este método acabou por não ser utilizado devido a mecânica que foi contruída á volta do jogo.

- `telaGameOver()`:

Este método desenha o ecrã de Game Over, onde mostra a pontuação que obteve.

- `touchStarted()`:

Este método é obrigatório por causa da Google, para por a musica a quando um clique.

Classe Inimigo

Na Classe Inimigo tenho os métodos:

- `movelInimigo()`:

Neste método é o que faz mostrar o inimigo e faz-lo mover-se. Neste metodo também chamamos o método `adoeceu()` e `refleteParede()`;

- `adoeceu()`:

Neste método verifica quanto tempo passou conforme cada inimigo e passando do intervalo faz o nível da doença subir, fazendo a imagem do inimigo também alterar, passando por duas fases, depois por último morre, causando ficar estático no local que morreu.

- `refleteParede()`:

Este método verifica se o inimigo bateu na parede e se sim, reflete.

- `queDoencaTenho()`:

Este método retorna a doença que tem.

- `mudarDoenca()`:

Este método faz com que o inimigo mude a doença que tenha e o seu tempo de escalação.

- `eliminaInimigo()`:

Este método faz que elimine o inimigo, apesar de não sequer ser usada.

Classe Player

Na classe player temos os seguintes métodos:

- `movePlayer()`:

Este método faz com que quando os estados das teclas serem pressionadas faça com que o player ande e mude a sua imagem para onde está virado.

- `rectRect()`:

Este método é o que verifica as colisões retornando “true” ou “false”.

- `refleteParedes()`:

Este método verifica se o player bateu na parede e se sim, reflete.

- `eliminaplayer()`:

Este método faz que elimine o player, apesar de não sequer ser usada.

- `mudalImagem()`:

Conforme a direção do player da ultima tecla muda a imagem para ficar correto.

- `iniciarPlayer()`:

Este método inicia o player.

Classe Seringa

Na classe Seringa temos os seguintes métodos:

- `atacaSeringa()`:

Este método é o que faz a seringa atacar, em que primeiro verifica se a mesma está a retornar onde se não, adiciona a velocidade de ataque á distancia que percorre a seringa verificando se tem alguma colisão com todos os inimigos, se atingiu verifica a doença que o quando o atinge e se a seringa estiver preparada para essa doença cura, se não não.

Ainda no método quando já a retornar, faz com que a seringa venha de volta.

- `rectRect()`:

Este método é o que verifica as colisões retornando “true” ou “false”.

- `moveSeringa()`:

Este método atualiza a posição da seringa com base na direção.

- `criaAudio()`:

Faz fazer o som de quando a seringa atinge um inimigo.

- `mudalImagemV()`:

Este método faz que conforme a direção do player a seringa mude de imagem para corresponder ao sentido para qual está o player.

- `pontuacaoDoenca()`:

Adiciona a pontuação do player o intervalo de tempo do inimigo dividido por 10 e arredondado.

- `iniciarSeringa()`:

Este método inicia a seringa.

Referências

1. p5js.org
2. Pedro Faria

Conclusão

O jogo final ficou dentro dos objetivos e correu bem. A parte mais complicada foi fazer a parte da colisão, mas no final, fiquei satisfeito com o resultado.

Ainda há coisas que poderiam ser melhoradas, como adicionar mais tipos de doenças ou melhorar os gráficos e a jogabilidade, possivelmente arranjar o facto de o player andar mais nas diagonais.